

BaCP: Backbone Contrastive Pruning for Preserving Representations in Extremely Sparse Neural Networks

Anonymous submission

Abstract

Neural network pruning is a powerful technique to compress models for deployment on resource-constrained devices. However, existing unstructured pruning methods fail to remain effective at extreme sparsity, e.g. at 95–99% sparsity levels, often suffering severe accuracy degradation due to the loss of critical feature representations. We propose Backbone Contrastive Pruning (BaCP), a novel pruning framework that maintains the network’s representational space under ultra-sparse constraints. Unlike prior pruning approaches that ignore lost knowledge, BaCP uses supervised and self-supervised contrastive objectives to align the sparse model’s embeddings with richer teacher models, thereby mitigating feature collapse. To produce highly sparse yet accurate backbones across a variety of models, BaCP makes use of multi-view augmentations and knowledge from three teacher sources: a pretrained model for task-agnostic features, a fine-tuned model for task-specific cues, and intermediate pruning snapshots for historical consistency. Further, BaCP is model-agnostic and can be combined with any iterative pruning strategy e.g. magnitude, movement, etc. Extensive experiments on image classification and language benchmarks demonstrate that BaCP consistently outperforms standard iterative pruning often by significant margins. Notably, BaCP-based pruned models even match or exceed the original dense model accuracy in some settings, highlighting an enhanced generalization under extreme compression. These results establish BaCP as an effective solution for preserving performance at extreme sparsity, opening the door to deployable deep networks with over 99% of weights removed. Our code is available at:

1 Introduction

Network pruning, the removal of redundant weights, has emerged as a promising solution to reduce model size and inference cost in order to make Deep neural networks (DNNs) practical for deployment on edge devices, mobile platforms, and real-time applications (Cheng, Zhang, and Shi 2024; Wang et al. 2025). Techniques range from one-shot magnitude pruning which remove the smallest-magnitude weights in one pass, to iterative schemes that gradually prune and retrain the model over multiple rounds (Han et al. 2015; Han, Mao, and Dally 2015). At moderate sparsity, many methods such as magnitude pruning (Han, Mao, and Dally 2015), SNIP (Lee, Ajanthan, and Torr 2018), movement pruning (Sanh, Wolf, and Rush 2020), and dynamic sparsification like Rigging the Lottery (Frankle et al. 2020; Evci et al.

2020), can prune networks with minor accuracy loss. However, achieving extreme sparsity beyond 95% remains highly challenging. Recent evaluations and surveys have shown that most unstructured pruning techniques break down at extreme sparsity levels, where accuracy drops precipitously (Blalock et al. 2020; Cheng, Zhang, and Shi 2024). Even state-of-the-art approaches (Li et al. 2024) struggle to preserve model performance when only a small fraction of weights remain. Crucially, these are the regimes where compression yields the greatest memory and speed benefits, so the inability to maintain performance above 95% sparsity is a major roadblock for real-world model compression.

When pruning aggressively, the core issue is that simply removing low-magnitude or supposedly *unimportant* weights ignores the broader network knowledge and feature structure. A deep model’s ability to generalize depends not just on individual large weights, but on the collective representation it has learned. Conventional pruning often disrupts the learned feature space, causing the pruned network to lose important task-agnostic knowledge acquired during pre-training. In other words, while focusing on preserving task-specific performance on the fine-tuning data, these methods forget the general features and invariances that the full model had, leading to catastrophic forgetting and overfitting in the pruned model. This problem is exacerbated at extreme sparsity; when 95–99% of weights are removed, feature representations collapse and gradients become unstable, often causing training to fail (Li et al. 2024).

To address these limitations, we propose Backbone Contrastive Pruning (BaCP), a framework designed specifically to maintain representational integrity under extreme sparsity. BaCP is inspired by recent advances in contrastive representation learning (Wu et al. 2018; Le-Khac, Healy, and Smeaton 2020), which learn robust features by pulling similar samples together in embedding space while pushing dissimilar samples apart. Contrastive learning has proven effective in producing transferable features in self-supervised and supervised settings, and has even been applied to pruning in the natural language processing (NLP) domain (Xu et al. 2022). Our key idea is to use contrastive objectives to preserve the model’s feature space during pruning. Instead of solely relying on the training labels and remaining weights to recover accuracy, BaCP supplies additional knowledge anchors for the pruned model via multiple contrastive signals which in-

clude: *pretrained model guidance* to provide task-agnostic knowledge by aligning the pruned model’s embeddings with those of the pretrained model which ensures the sparse network retains general features and doesn’t deviate far from the rich representation of its ancestor, *fine-tuned model guidance* as a teacher signal to impart task-specific knowledge which helps the pruned model preserve important discriminative features, and *snapshot self-consistency* to enforce temporal consistency and retain historical knowledge that might otherwise vanish at higher sparsity levels. By combining these signals, BaCP effectively acts as a form of contrastive knowledge distillation for pruning, directly targeting the root cause of performance collapse: rather than purely optimizing for the task loss which can overfit on few weights, we explicitly maintain the backbone representations that underpin generalization. Our contributions are summarized as follows:

- We introduce a novel pruning framework, BaCP, that integrates contrastive learning during the pruning process to sustain learning capacity at extreme sparsity, up to 99% weights pruned. To our knowledge, BaCP is the first to apply contrastive representation alignment to vision models under such high sparsity.
- Our approach is model-agnostic and pruning-criterion-agnostic. We show it can be flexibly combined with various unstructured pruning strategies. BaCP does not alter the pruning criteria or require any special architecture changes; it adds a training paradigm that consistently boosts accuracy on top of standard pruning.
- We conduct extensive experiments on multiple computer vision and NLP benchmarks to demonstrate that BaCP consistently outperforms traditional iterative pruning, especially in the high-sparsity regime.

2 Related Work

Unstructured Pruning and Extreme Sparsity: Classic approaches prune weights with the smallest magnitudes, reflecting the intuition that low-magnitude weights contribute least to the output. This simple magnitude pruning strategy was popularized by (Han, Mao, and Dally 2015) and remains a strong baseline. Subsequent improvements include pruning at initialization such as SNIP (Lee, Ajanthan, and Torr 2018), GraSP (Wang, Zhang, and Grosse 2020) which identify important connections before training, and dynamic pruning/growing methods during training, e.g. RigL (Evci et al. 2020) which periodically reintroduces weights based on gradient flow. Iterative pruning with retraining has been shown to yield better results than one-shot pruning, especially under high sparsity, as the network can recover and adapt after each pruning step. Despite success at moderate sparsity, most methods struggle when pushing beyond 90–95% sparsity. Blalock et al. (Blalock et al. 2020) provide an extensive survey, concluding that the supposed “state of the art” methods often barely outperform magnitude pruning and fail to maintain accuracy at extreme sparsities. A more recent survey (Cheng, Zhang, and Shi 2024) echoes this, highlighting significant accuracy degradation past 95% sparsity and identifying it as a key open challenge. There is also a line of research on the Lottery Ticket Hypothesis (LTH), which

postulates that within a dense network there exist sparse sub-networks termed as “winning tickets” that can be trained to full accuracy if isolated with the right initialization (Frankle et al. 2020). However, identifying tickets at 98–99% sparsity remains difficult; many layers in deep networks cannot be pruned so aggressively without retraining failing.

Pruning in Vision Models vs. Language Models: While core pruning techniques can be applied across domains, there are notable differences between convolutional neural network (CNN) based models and Transformer-based models in terms of pruning behavior. At extreme sparsity, CNN can suffer from layer collapse; if an early convolutional layer is pruned too heavily, feature maps become too degraded. Recent extreme pruning results on ImageNet classifiers show that careful layer-wise sparsity schedules and other techniques like cyclic regrowth are needed to push ResNet-50 beyond 99% sparsity (Li et al. 2024).

Transformers have multiple component types including self-attention heads, feed-forward layers, embedding layers, and pruning can target entire heads or neurons in a structured fashion or individual weights in the matrices, following an unstructured way. Studies have that many attention heads in BERT can be removed with little impact, implying redundancy (Michel, Levy, and Neubig 2019). However, unstructured pruning of Transformer models during fine-tuning often requires careful techniques. Movement pruning (Sanh, Wolf, and Rush 2020) was introduced to prune BERT during fine-tuning by using the sign of weight updates. Another approach pruned entire blocks of Transformers using importance scores (Lagunas et al. 2021). Lottery ticket phenomena in BERT has also been examined (Prasanna, Rogers, and Rumshisky 2020), finding all tickets can win with proper retraining. For large-scale LLMs with billions of parameters, recent works like WANDA (Sun et al. 2023) have focused on pruning without any retraining, by exploiting activation patterns. Our work includes experiments on smaller Transformers to test BaCP in the NLP setting. We found that Transformers are particularly sensitive to extreme pruning. For instance, unstructured magnitude pruning of RoBERTa at 99% sparsity often fails to even fine-tune properly. Using BaCP, the pruned RoBERTa models were able to retain much higher accuracy.

Contrastive Pruning and Knowledge Retention: Our work is inspired by the method the contrastive pruning (CAP) method (Xu et al. 2022) where the authors pruned BERT-like language models while using contrastive losses to align the pruned model with the original pretrained and fine-tuned models. CAP demonstrates that task-agnostic knowledge from the pretraining stage is indeed vital to the extent that without it, pruned models suffer from representational collapse. CAP was primarily validated on NLP tasks and on Transformers, with the intuition that pretrained language models encode a lot of general linguistic knowledge that should be preserved. Our work extends this concept to vision models, which also benefit from preserving both general and task-specific feature structure when pruned. We show that even when pruning image classifiers or smaller language models, the contrastive alignment approach yields consistent improvements.

3 BaCP: Methodology

BaCP is a general pruning framework that augments any iterative pruning process with contrastive learning to preserve the model’s representations. The overall pipeline is illustrated in Figure 1. Below, we describe the components of BaCP in a structured manner.

3.1 Preliminary: Iterative Unstructured Pruning

Unstructured pruning removes individual weights from the network by setting them to zero, typically guided by an importance score. Formally, let W denote the set of model weights. Pruning produces a binary mask M of the same shape as W , and the pruned model uses $W \odot M$ where $\odot$ represents the Hadamard product, as its weights. A zero in M indicates the corresponding weight is pruned. In iterative pruning, we repeatedly update M over training epochs to achieve the final sparsity level. In our proposed BaCP, we integrate three popular unstructured criteria:

- **Magnitude Pruning:** We compute an importance score $I_w = |w|$ for each weight. At a given pruning step, a global threshold θ is chosen such that a desired fraction of weights have $|w| < \theta$. Those weights are then pruned.
- **SNIP-it Pruning:** In SNIP-it pruning (Verdenius, Stol, and Forré 2020), weights are scored by their magnitude times the current gradient, $I_w = \left| \frac{\partial \mathcal{L}}{\partial w} \cdot w \right|$. We prune the weights with the smallest $|w \cdot \nabla_w|$ scores at each step.
- **WANDA Pruning:** It prunes weights based on the product of weight magnitude and input activation norm. For a given weight w_{ij} connecting neuron i to j , WANDA’s criterion is $I_{ij} = |w_{ij}| \cdot |x_i|_2$, where $|x_i|_2$ is the ℓ_2 norm of the input activation to that weight.

3.2 Multi-View Data Augmentation

In order to use contrastive framework, we need to create multiple views of each input to serve as positive pairs for contrastive loss. Given a dataset $\mathcal{D} = (x_i, y_i)_{i=1}^N$ of samples and labels, we generate two stochastic augmentations of each sample: $\tilde{x}_i^{(1)} = \text{Aug}^{(1)}(x_i)$ and $\tilde{x}_i^{(2)} = \text{Aug}^{(2)}(x_i)$. Here $\text{Aug}^{(j)}(\cdot)$ is a j -th data augmentation function that applies random transformations appropriate for the domain. For images, we use random cropping, flips, color jitter, auto-augment policies, etc., while for text we do not apply any contrastive text-based augmentations. The two views $\tilde{x}_i^{(1)}, \tilde{x}_i^{(2)}$ preserve the underlying content of x_i but appear different to the model, which is crucial for contrastive learning to discover invariant features. We construct an augmented dataset $\tilde{\mathcal{D}} = (\tilde{x}_i^{(1)}, \tilde{x}_i^{(2)}, y_i)_{i=1}^N$. In each training iteration, whenever we sample a datapoint, we use both augmented views. One view is fed to the current model while the other view is fed to a reference model’s for computing contrastive alignment.

3.3 Encoder and Projection Architecture

We utilize a standard architecture for contrastive learning consisting of an encoder and a projection head. The encoder $f_{\text{enc}}(x; W)$ with weights W , is essentially the backbone network being pruned. The encoder produces a high-dimensional

feature representation vector as: $\mathbf{r} = f_{\text{enc}}(\tilde{x}; W) \in \mathbb{R}^{D_E}$, where D_E is the dimension of the encoder’s output. We will refer to $\mathbf{r}$ as the embedding or representation of the input. This representation $\mathbf{r}$ is then passed through a smaller projection head $g(\cdot)$, which is typically a 1- or 2-layer neural network, often a multilayer perceptron (MLP). The projection head maps the representation to a lower-dimensional space where the contrastive loss is applied: $\mathbf{z} = g(\mathbf{r}) \in \mathbb{R}^{D_P}$. We perform ℓ_2 normalization on $\mathbf{z}$ before computing contrastive losses.

The rationale for having a separate projection head is twofold: first, it provides an additional space where contrastive learning objectives can be applied without directly altering the representational space used by the final task. In practice, after we finish BaCP training and obtain a pruned model, we discard $g(\cdot)$ and only fine-tune the encoder on the task with a standard classification head. Second, the projection can reduce dimensionality, which makes contrastive computation more efficient. Next, we describe the loss functions used to compare these $\mathbf{z}$ vectors.

3.4 Contrastive Loss Functions

We employ two types of contrastive loss: supervised contrastive loss for labeled data and self-supervised contrastive loss for unlabeled pairing.

Supervised Contrastive Loss. In a given batch, for an anchor sample x_i with label y_i , the set of positive indices $P(i)$ is defined as all other samples in the batch that have the same label $y_j = y_i$ with $j \neq i$. The negatives are all samples with different labels. Let $\mathbf{z}_i$ be the projected embedding of sample x_i . The supervised contrastive loss is given by

$$L_i^{\text{sup}} = \frac{1}{|P(i)|} \sum_{j \in P(i)} -\log \frac{\exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_j)}{\tau}\right)}{\sum_{k=1}^{N_{\text{batch}}} \mathbf{1}_{[k \neq i]} \exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_k)}{\tau}\right)}$$

where $\text{sim}(\mathbf{z}_i, \mathbf{z}_j) = \frac{\mathbf{z}_i^T \mathbf{z}_j}{\|\mathbf{z}_i\| \|\mathbf{z}_j\|}$ is the cosine similarity between two ℓ_2 -normalized vectors, and τ is a temperature hyperparameter controlling the sharpness of the distribution. Intuitively, this loss L_i^{sup} encourages $\mathbf{z}_i$ to be close to all positive $\mathbf{z}_j$ and far from all others.

Self-Supervised Contrastive Loss. Suppose for a particular sample x_i , we have two augmented views indexed as i and j . We treat (i, j) as a positive pair. The self-supervised contrastive loss for this pair is

$$L_{i,j}^{\text{self}} = -\log \frac{\exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_j)}{\tau}\right)}{\sum_{k=1}^{2N_{\text{batch}}} \mathbf{1}_{[k \neq i]} \exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_k)}{\tau}\right)},$$

where the batch size is effectively $2N$. Theoretically, we should compute this loss for both i and j as anchors and take the average. But in practice we only compute it once over the entire $2N$ batch ensuring both directions are handled. This encourages two views of the same sample to be highly similar, while ensuring that views of different samples are dissimilar.

Algorithm 1: BaCP Training Framework

Require: Dataset $D = \{x_{1i}, x_{2i}; y_i\}_{i=1}^N$, Current model f_{curr} , Pretrained model f_{pre} , Finetuned model f_{fine} , Snapshots $S = \{\}$, Loss weights $\lambda_{CE}, \lambda_{PrC}, \lambda_{FiC}, \lambda_{SnC}$, Pruner P , Recovery epochs T_{rec}

Ensure: Trained sparse model f_{curr}

```

1: for epoch = 1 to  $T$  do
2:   for each batch  $(x_1, x_2; y)$  in  $D$  do
3:      $r_{\text{curr}} \leftarrow f_{\text{curr}}(x_1, \text{extract\_features} = \text{True})$ 
4:      $z_{\text{curr}} \leftarrow f_{\text{curr}}.\text{projection\_head}(r_{\text{curr}})$ 
5:      $z_{\text{pre}} \leftarrow f_{\text{pre}}(x_2)$   $\triangleright$  No gradient
6:      $z_{\text{fine}} \leftarrow f_{\text{fine}}(x_2)$   $\triangleright$  No gradient
7:      $\text{logits} \leftarrow \text{classification\_head}(r_{\text{curr}})$ 
8:      $\mathcal{L}_{CE} \leftarrow \text{CrossEntropyLoss}(\text{logits}, y)$ 
9:      $\mathcal{L}_{PrC} \leftarrow \mathcal{L}_{PrC}^{\text{sup}} + \mathcal{L}_{PrC}^{\text{self}}$ 
10:     $\mathcal{L}_{FiC} \leftarrow \mathcal{L}_{FiC}^{\text{sup}} + \mathcal{L}_{FiC}^{\text{self}}$ 
11:     $\mathcal{L}_{SnC} \leftarrow 0$ 
12:    for each  $\phi_{\text{snap}} \in S$  do
13:       $E_{\text{snap}} \leftarrow f_{\text{snap}}(x_2)$   $\triangleright$  No gradient
14:       $\mathcal{L}_{SnC} \leftarrow \mathcal{L}_{SnC} + \mathcal{L}_{SnC}^{\text{sup}} + \mathcal{L}_{SnC}^{\text{self}}$ 
15:    end for
16:     $\mathcal{L}_{\text{total}} \leftarrow \lambda_{CE}\mathcal{L}_{CE} + \lambda_{PrC}\mathcal{L}_{PrC} + \lambda_{SnC}\mathcal{L}_{SnC} + \lambda_{FiC}\mathcal{L}_{FiC}$ 
17:    Weight update step
18:     $P.\text{prune}(f_{\text{curr}}, P.\text{apply\_mask}(f_{\text{curr}}))$ 
19:  end for
20:  for recovery_epoch = 1 to  $T_{\text{rec}}$  do
21:    for each batch  $(x_1, x_2; y)$  in  $D$  do
22:      Repeat steps 3–17
23:     $P.\text{apply\_mask}(\phi_{\text{curr}})$   $\triangleright$  Maintain sparsity
24:  end for
25: end for
26:  $S \leftarrow S \cup \{\text{deepcopy}(f_{\text{curr}})\}$   $\triangleright$  Save model state
27: end for
28: return  $f_{\text{curr}}$   $\triangleright$  Final sparse model

```

3.5 Multi-Objective Loss Modules

We now introduce the key modules of BaCP. At any iteration, we have the following entities:

- The current sparse model consists of an encoder f_{curr} and a projection head g_{curr} , with weights W_{curr} that are mostly pruned.
- A pretrained model composed of encoder f_{pre} and projection g_{pre} , representing the original model before task-specific fine-tuning. This model captures task-agnostic, general-purpose knowledge.
- A fine-tuned model consisting of encoder f_{fine} and projection g_{fine} . This is a dense model fine-tuned on the target task. It serves as the performance upper bound for the task.
- A set of snapshot models $\{f_{\text{snap}}^{(1)}, f_{\text{snap}}^{(2)}, \dots, f_{\text{snap}}^{(K)}\}$ taken at intermediate pruning stages (e.g., after reaching 50% sparsity, 75% sparsity, etc.), each paired with its own projection head $g_{\text{snap}}^{(k)}$. These snapshots record the model's pruned states over time.

BaCP defines four loss components computed using these models' outputs:

Cross-Entropy (CE) Loss Module: We feed one view of each sample, say, $\tilde{x}_i^{(1)}$ into the current model's encoder, then pass the resulting representation $\mathbf{r}$ through a classification head to predict the label $\hat{y}$. We compute the usual cross-entropy loss $\mathcal{L}_{CE}$ between $\hat{y}$ and the ground truth label y . Meanwhile, the same representation $\mathbf{r}$ is also passed through the current model's projection head to produce $\mathbf{z}_{\text{curr}}$, which will be used in the contrastive losses described next.

Pretrained Contrastive (PrC) Loss Module: We feed the other view of each sample $\tilde{x}_i^{(2)}$ into the pretrained model's encoder f_{pre} and obtain its projected embedding $\mathbf{z}_{\text{pre}}$. We then compare $\mathbf{z}_{\text{curr}}$ with $\mathbf{z}_{\text{pre}}$ using both supervised and self-supervised contrastive losses. Conceptually, we treat $\mathbf{z}_{\text{pre}}$ as an alternative view of the same input. The PrC loss $\mathcal{L}_{PrC}$ is defined as the sum of a supervised contrastive loss and a self-supervised contrastive loss between $\mathbf{z}_{\text{curr}}$ and $\mathbf{z}_{\text{pre}}$ representations:

$$\mathcal{L}_{PrC} = \mathcal{L}_{PrC}^{\text{sup}}(\mathbf{z}_{\text{curr}}, \mathbf{z}_{\text{pre}}) + \mathcal{L}_{PrC}^{\text{self}}(\mathbf{z}_{\text{curr}}, \mathbf{z}_{\text{pre}})$$

The effect of this loss is to pull the current model's embedding distribution closer to that of the original pretrained network.

Fine-tuned Contrastive (FiC) Loss Module: This module is analogous to PrC, but uses the fine-tuned task-specific model as the reference. We pass view 2 through the fine-tuned full model f_{fine} to obtain the projected embedding $\mathbf{z}_{\text{fine}}$. We then contrast $\mathbf{z}_{\text{curr}}$ and $\mathbf{z}_{\text{fine}}$ using both supervised and self-supervised contrastive losses as

$$\mathcal{L}_{FiC} = \mathcal{L}_{FiC}^{\text{sup}}(\mathbf{z}_{\text{curr}}, \mathbf{z}_{\text{fine}}) + \mathcal{L}_{FiC}^{\text{self}}(\mathbf{z}_{\text{curr}}, \mathbf{z}_{\text{fine}})$$

The fine-tuned model can be thought of as a powerful teacher that knows how to solve the task with high accuracy.

Snapshot Contrastive (SnC) Loss Module: The snapshot contrastive loss aligns the current model with all of its past selves. Each snapshot $f_{\text{snap}}^{(k)}$ with its corresponding projection head $g_{\text{snap}}^{(k)}$ can generate an embedding $\mathbf{z}_{\text{snap}}^{(k)}$ for the input. We use $\tilde{x}^{(2)}$ for these snapshots. For each snapshot k , we compute supervised and self-supervised contrastive losses between $\mathbf{z}_{\text{curr}}$ and $\mathbf{z}_{\text{snap}}^{(k)}$. The overall snapshot contrastive loss $\mathcal{L}_{SnC}$ sums these over all K snapshots as

$$\mathcal{L}_{SnC} = \sum_{k=1}^K \left(\mathcal{L}_{SnC,k}^{\text{sup}}(\mathbf{z}_{\text{curr}}, \mathbf{z}_{\text{snap}}^{(k)}) + \mathcal{L}_{SnC,k}^{\text{self}}(\mathbf{z}_{\text{curr}}, \mathbf{z}_{\text{snap}}^{(k)}) \right)$$

3.6 Final Objective and BaCP Training

We combine the above loss components into a single multi-task objective that the current model optimizes. The total loss $\mathcal{L}$ is a weighted sum as

$$\mathcal{L} = \lambda_{CE} \mathcal{L}_{CE} + \lambda_{PrC} \mathcal{L}_{PrC} + \lambda_{FiC} \mathcal{L}_{FiC} + \lambda_{SnC} \mathcal{L}_{SnC},$$

where $\lambda_{CE}, \lambda_{PrC}, \lambda_{FiC}, \lambda_{SnC}$ are hyperparameters used to balance the contributions of the respective components. We keep the pretrained and fine-tuned models fixed as they serve only

to provide target embeddings. The snapshots are earlier versions of the current model; we don’t update them either once saved. The current model is the only one being updated by the gradients.

We begin by initializing a dense pretrained model f_{pre} , and also prepare a fine-tuned version f_{fine} trained on the target task. The current model f_{curr} is set to f_{pre} , serving as the starting point for pruning. The BaCP procedure then proceeds iteratively: at each sparsity level, we perform training over multiple epochs. In each epoch, two augmented views of each input sample are generated, and the current model is trained using a combination of four losses, computed using embeddings from the current and reference models. Only the current model’s weights W_{curr} are updated, while a binary mask M ensures pruned weights remain zero. At the end of each epoch, additional weights are pruned according to a chosen criterion (e.g., magnitude or SNIP-it), and a snapshot of the current model may be saved for use in SnC during future iterations. This cycle continues, progressively increasing sparsity and maintaining alignment with both general and task-specific representations. Once the target sparsity is reached, the contrastive projection head g_{curr} is removed, and a linear classifier is trained on top of the encoder’s features. Complete algorithm detailing main steps is presented in Appendix.

4 Experimental Results

4.1 Experimental Setup

Datasets and Models We evaluate our framework across a diverse range of model architectures and tasks. For image classification task, we use CIFAR-10, CIFAR-100, MNIST, Fashion-MNIST, Extended-MNIST, and SVHN datasets. Here we present results for CIFAR-10 only whereas Appendix details the results for rest of the datasets.

For CNN based models, we have used: ResNet-50 and ResNet-101, which represent varying depths of the ResNet family, as well as VGG-11 and VGG-19, which provide simpler yet deeper CNN baselines. All CNNs operate on input images resized to 32×32 and use the AutoAugment policy during contrastive training.

For transformer-based vision models, we incorporate ViT-Tiny and ViT-Small, both applied to CIFAR-10 with input images resized to 224×224. These compact Vision Transformers offer a trade-off between performance and computational efficiency, allowing us to explore contrastive pruning on attention-based architectures.

For language tasks, we utilize DistilBERT and RoBERTa as representative language models. For datasets, we choose SST-2 dataset which is used for binary sentiment classification for language models, testing their ability to capture fine-grained sentiment features, and WikiText-2 dataset which is used for the task of Masked Language Modeling (MLM), testing the models generalization and sequence modeling performance after pruning.

Baselines for Comparison and Evaluation Metrics: We evaluate BaCP by comparing it against traditional iterative pruning and several strong baselines under identical training protocols. As an upper bound, we first train each model

to convergence without any pruning to obtain a dense baseline. For sparse baselines, we adopt a standard supervised iterative pruning-retraining approach under three high sparsity regimes: 95%, 97%, and 99%. Within this framework, we explore several pruning strategies: magnitude pruning, SNIP-it pruning and WANDA pruning. Although originally designed for causal LLMs in zero-shot settings, we adapt WANDA for iterative pruning with retraining, applying it to tasks like sequence classification and Masked Language Modelling (MLM). For all classification tasks, we report Top-1% accuracy. For the language modeling task, we report both classification accuracy for masked token prediction and perplexity to evaluate generative quality. In all experiments, we track the final sparsity ratio and measure model performance across sparsity levels of 95%, 97%, and 99%. Additional results and ablation studies are provided in Appendix.

Training Protocol: Across all experiments, CNN and ViT dense baselines are trained using Stochastic Gradient Descent (SGD) with a learning rate of 0.01 while LLMs use AdamW with a learning rate of 0.00002, both with a linear learning rate scheduler with warm-up, and appropriate batch sizes and epochs to ensure convergence. For CNNs, sparse baselines follow the same optimizer and learning rate as the dense models. ViTs and LLMs use AdamW with a learning rate of 0.001 and 0.00005 respectively. For BaCP, we adopt a two-phase training strategy: CNNs and ViTs are first pretrained using SGD (with a higher learning rate of 0.1 CNNs and 0.1-0.01 for ViTs) while LLMs use AdamW (with a higher learning rate of 0.00001), followed by fine-tuning using the AdamW optimizer with a learning rate of 0.0001 and 0.00002 respectively on the downstream task. For pruning, we employ a 5–10 stage iterative pruning-retraining protocol, where models are pruned once at the start of each epoch. To control sparsity over iterations, we adopt the cubic sparsity scheduler from (Zhu and Gupta 2017), adapted for epoch-wise pruning instead of step-wise pruning.

4.2 Main Results

We conduct a comprehensive evaluation of BaCP across multiple model architectures, datasets, pruning strategies, and sparsity levels. Datasets with fewer classes, such as CIFAR-10, consistently yield superior results, particularly across ResNet and VGG architectures. In contrast, VGG-11 and VGG-19 underperform on CIFAR-100, while ResNet models maintain relatively strong results. Vision Transformers (ViTs) trained with BaCP show competitive—and often improved—accuracy, with performance gains becoming more pronounced as sparsity increases from 0.95 to 0.99. VGGs struggle more under BaCP, which we attribute to architectural differences such as the absence of skip connections.

This performance advantage becomes increasingly evident at higher sparsity levels. In convolutional architectures like ResNet and VGG, BaCP yields consistent improvements across various pruning methods, including Magnitude, SNIP-it, and Wanda. These improvements are particularly significant under high sparsity, where the parameter budget is severely constrained. While Iterative Pruning (I.P.) remains competitive at lower sparsity, it degrades more rapidly under

Table 1: Pruning Accuracy (%) Comparison at Different Sparsity Levels Across Models, Datasets, and Pruning Types

Model	Dataset	Dense	95% Sparsity						97% Sparsity						99% Sparsity					
			Magnitude		SNIP-it		WANDA		Magnitude		SNIP-it		WANDA		Magnitude		SNIP-it		WANDA	
			I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP
ResNet-50	CIFAR-10	93.02	91.97	93.58	90.77	93.12	91.78	93.15	91.39	93.27	90.58	92.71	90.81	93.02	89.87	92.32	88.18	91.9	86.59	90.87
	CIFAR-100	73.33	70.43	71.46	66.57	69.57	67.73	70.15	69.27	70.49	67.01	68.79	65.82	67.51	65.13	67.06	62.13	64.06	55.18	58.81
ResNet-101	CIFAR-10	91.80	90.43	93.23	90.61	93.75	90.09	93.09	90.22	92.90	89.86	92.64	89.67	92.75	89.51	92.70	87.83	92.20	86.46	91.92
	CIFAR-100	74.58	71.12	72.16	68.10	71.03	68.26	70.05	69.41	72.67	66.86	70.39	66.94	70.07	66.27	70.23	63.45	66.17	58.46	62.61
VGG-11	CIFAR-10	89.94	90.10	90.44	89.60	90.37	89.49	90.30	89.61	90.65	89.64	90.46	89.04	90.23	88.97	90.44	88.24	90.11	88.28	89.06
	CIFAR-100	67.43	65.89	64.31	65.73	62.88	64.61	61.37	65.32	64.33	65.47	63.43	63.91	61.41	65.11	62.76	63.41	61.43	61.5	57.93
VGG-19	CIFAR-10	91.78	91.53	92.57	91.52	92.47	91.46	92.18	91.66	92.60	91.34	91.40	90.90	91.80	90.56	92.12	89.74	84.76	90.50	91.79
	CIFAR-100	69.54	68.70	68.00	68.03	67.28	67.47	66.99	68.04	68.52	67.16	66.83	67.48	65.74	66.60	66.88	62.75	64.16	65.68	62.69
ViT-Tiny	CIFAR-10	97.46	85.88	91.57	84.46	90.43	72.48	85.59	84.12	89.78	82.64	88.40	72.04	83.19	74.21	87.01	77.60	85.00	69.79	81.25
	CIFAR-100	85.15	91.19	92.22	91.93	88.84	88.14	87.78	90.58	92.39	90.72	89.28	84.86	90.02	86.28	90.67	88.11	89.07	80.32	81.78
ViT-Small	CIFAR-10	98.40	91.19	92.22	91.93	88.84	88.14	87.78	90.58	92.39	90.72	89.28	84.86	90.02	86.28	90.67	88.11	89.07	80.32	81.76
	CIFAR-100	90.85	73.54	74.65	73.59	73.27	66.01	66.38	70.42	70.04	69.90	70.66	58.48	59.83	61.24	63.88	64.03	64.17	31.91	44.39
DistilBERT	SST-2	91.11	82.57	85.46	82.09	84.50	81.97	84.86	81.25	83.29	80.05	84.25	79.93	82.21	80.53	82.33	79.93	81.25	80.41	81.61
	WikiText-2	64.44	47.98	49.79	45.71	47.33	45.65	47.29	45.93	47.19	42.48	44.99	43.57	46.03	41.34	43.68	37.65	41.05	40.85	42.57
RoBERTa	SST-2	94.83	81.85	83.41	87.38	87.86	82.45	84.86	80.77	83.17	85.82	86.06	81.49	83.05	78.12	82.57	81.41	84.38	80.29	82.09
	WikiText-2	73.16	53.95	55.18	52.85	52.74	51.54	52.32	51.59	53.27	50.43	50.92	25.97	50.85	46.39	51.21	44.77	46.73	18.30	47.10

aggressive pruning, whereas BaCP retains substantially more predictive capacity.

Transformer-based architectures benefit even more from BaCP. Models such as ViT, DistilBERT, and RoBERTa show notable accuracy improvements, up to 2 percentage points, when pruned using BaCP compared to I.P. This underscores the role of contrastive supervision in preserving useful representational structures, especially in pruning-sensitive models.

An exception arises with deeper convolutional networks like VGG-19 on complex image datasets. In some cases, I.P. slightly outperforms BaCP at moderate sparsity levels. We hypothesize that this is due to architectural factors such as the lack of residual connections along with the depth of the model.

Among the pruning strategies evaluated, Magnitude pruning consistently provides the strongest overall performance for both BaCP and I.P., followed by SNIP-it. Wanda, in contrast, tends to be less stable and often underperforms under high sparsity, especially on more complex tasks.

In summary, BaCP emerges as a robust and generalizable pruning framework that enhances sparsity resilience across diverse architectures and tasks. These findings highlight the importance of contrastive learning in structured pruning and point toward future directions for mitigating performance degradation under extreme model compression.

References

- Blalock, D.; Gonzalez Ortiz, J. J.; Frankle, J.; and Gutttag, J. 2020. What is the state of neural network pruning? *Proceedings of machine learning and systems*, 2: 129–146.
- Cheng, H.; Zhang, M.; and Shi, J. Q. 2024. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Evci, U.; Gale, T.; Menick, J.; Castro, P. S.; and Elsen, E. 2020. Rigging the lottery: Making all tickets winners. In *International conference on machine learning*, 2943–2952. PMLR.
- Frankle, J.; Dziugaite, G. K.; Roy, D.; and Carbin, M. 2020. Linear mode connectivity and the lottery ticket hypothesis. In *International Conference on Machine Learning*, 3259–3269. PMLR.
- Han, S.; Mao, H.; and Dally, W. J. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *arXiv preprint arXiv:1510.00149*.
- Han, S.; Pool, J.; Tran, J.; and Dally, W. 2015. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems*, 28.
- Lagunas, F.; Charlaix, E.; Sanh, V.; and Rush, A. M. 2021. Block pruning for faster transformers. *arXiv preprint arXiv:2109.04838*.
- Le-Khac, P. H.; Healy, G.; and Smeaton, A. F. 2020. Contrastive representation learning: A framework and review. *Ieee Access*, 8: 193907–193934.
- Lee, N.; Ajanthan, T.; and Torr, P. H. 2018. Snip: Single-shot network pruning based on connection sensitivity. *arXiv preprint arXiv:1810.02340*.

- Li, A.; Durrant, A.; Markovic, M.; Huang, T.; Kundu, S.; Chen, T.; Yin, L.; and Leontidis, G. 2024. Pushing the Limits of Sparsity: A Bag of Tricks for Extreme Pruning. *arXiv preprint arXiv:2411.13545*.
- Michel, P.; Levy, O.; and Neubig, G. 2019. Are sixteen heads really better than one? *Advances in neural information processing systems*, 32.
- Prasanna, S.; Rogers, A.; and Rumshisky, A. 2020. When BERT plays the lottery, all tickets are winning. *arXiv preprint arXiv:2005.00561*.
- Sanh, V.; Wolf, T.; and Rush, A. 2020. Movement pruning: Adaptive sparsity by fine-tuning. *Advances in neural information processing systems*, 33: 20378–20389.
- Sun, M.; Liu, Z.; Bair, A.; and Kolter, J. Z. 2023. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*.
- Verdenius, S.; Stol, M.; and Forré, P. 2020. Pruning via Iterative Ranking of Sensitivity Statistics. *arXiv:2006.00896*.
- Wang, C.; Zhang, G.; and Grosse, R. 2020. Picking winning tickets before training by preserving gradient flow. *arXiv preprint arXiv:2002.07376*.
- Wang, X.; Tang, Z.; Guo, J.; Meng, T.; Wang, C.; Wang, T.; and Jia, W. 2025. Empowering edge intelligence: A comprehensive survey on on-device ai models. *ACM Computing Surveys*, 57(9): 1–39.
- Wu, Z.; Xiong, Y.; Yu, S. X.; and Lin, D. 2018. Unsupervised feature learning via non-parametric instance discrimination. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 3733–3742.
- Xu, R.; Luo, F.; Wang, C.; Chang, B.; Huang, J.; Huang, S.; and Huang, F. 2022. From dense to sparse: Contrastive pruning for better pre-trained language model compression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, 11547–11555.
- Zhu, M.; and Gupta, S. 2017. To prune, or not to prune: exploring the efficacy of pruning for model compression. *arXiv preprint arXiv:1710.01878*.

Appendix

A. Training Protocol

Convolutional Neural Networks. For convolutional backbones (ResNet-50, ResNet-101, VGG-11, and VGG-19), we pretrain using Stochastic Gradient Descent (SGD) with a learning rate of 0.1. However, for VGG-19, the learning rate was reduced to 0.05 to prevent gradient explosion due to its depth. All models are finetuned using AdamW with a lower learning rate of $1e-4$ for stable task adaptation. We prune at the beginning of each epoch for 5 epochs and allow 10 epochs for recovery post-pruning.

Table 2: BaCP Hyperparameters for Convolutional Neural Networks

Hyperparameter	ResNet-50/101	VGG-11/19
Dataset	CIFAR-10/100	
Pretraining Optimizer and LR	SGD(0.1)	SGD(0.1) / SGD(0.05)
Finetuning Optimizer and LR	AdamW($1e-4$)	
Epochs	5	
Recovery Epochs	10	
Batch Size	512	

Vision Transformers. For vision transformers (ViT-Tiny and ViT-Small), We maintain the same pre-training and finetuning strategy used for CNNs. Both models are pretrained using SGD with a learning rate of 0.1 and finetuned using AdamW with a learning rate of $1e-4$.

Table 3: BaCP Hyperparameters for Vision Transformers

Hyperparameter	ViT-Tiny	ViT-S
Dataset	CIFAR-10/100	
Pretraining Optimizer and LR	SGD(0.1)	
Finetuning Optimizer and LR	AdamW($1e-4$)	
Epochs	5	
Recovery Epochs	10	
Batch Size	256	

Language Models. For DistilBERT and RoBERTa, we use a learning rate of $5e-5$ and $1e-5$, respectively, for pretraining on the SST-2 dataset. Both models are then finetuned using AdamW with a learning rate of $1e-4$. When pretraining on the WikiText-2 dataset, we use AdamW with a learning rate of $1e-3$ for DistilBERT and $5e-4$ for RoBERTa.

Table 4: BaCP Hyperparameters for Language Models

Hyperparameter	DistilBERT	RoBERTa
Dataset	SST-2 / WikiText-2	
Pretraining Optimizer and LR	AdamW($5e-5$) / AdamW($1e-3$)	AdamW($1e-5$) / AdamW($5e-4$)
Finetuning Optimizer and LR	AdamW($1e-4$)	
Batch Size	64	

B. Ablation Study

To assess the contribution of each module in the proposed BaCP framework, we conduct a thorough ablation study. The framework includes four loss components weighted by hyperparameters: λ_{CE} , λ_{PrC} , λ_{SnC} , and λ_{FiC} . We disable modules by setting their corresponding λ to zero.

We also compare supervised vs. unsupervised contrastive training and vary the temperature τ to study sensitivity.

B.1 Multi-Objective Loss Weights To balance the contributions of each loss component, we initially set the loss weights $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ to a static value of 0.25. This gives equal importance to each loss module in the total loss computation. These static lambdas represent a simple yet effective approach where all objects are treated with equal priority. However, static weighting may not always be optimal. To investigate this further, we explore the effect on training with learnable parameters, allowing the model to dynamically adjust the weights of each loss component during training. This introduces flexibility and lets the model prioritize objectives that are more critical for convergence and/or performance under high sparsity constraints. Additionally, manual tuning of lambda values is also feasible, offering greater user control based on task specific insights.

In this experiment, we compare performance under static and learnable settings analyze the impact of lambda configuration. The results can be seen in table 5. When the lambda parameters were learnable, we recorded their final values after training and tracked their progression throughout the training process. Figure ?? illustrates how the lambdas evolved over time.

Table 5: Impact of Static vs Learnable Lambda Parameters on Accuracy at Varying Sparsity Levels

Lambda Type	λ_1	λ_2	λ_3	λ_4	Accuracy (%) at Sparsity		
					0.95	0.97	0.99
Static (Current)	0.25	0.25	0.25	0.25			
Learnable Parameters	–	–	–	–			

B.2 Module Importance We also want to examine how each loss module plays a part in the overall performance of the model. To this end, we 0 out their weights, effectively removing the loss from updating the model. We run experiments showcased in table 6

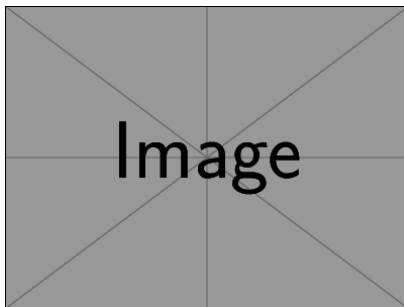
Table 6: Ablation: Effect of Removing Modules ($\lambda_i = 0$).

λ_1 (CE)	λ_2 (PrC)	λ_3 (SnC)	λ_4 (FiC)	Modules Used	Acc. (%)
✓	✓	✓	✓	BaCP (Current)	–
×	✓	✓	✓	no CE	–
✓	×	✓	✓	no PrC	–
✓	✓	×	✓	no SnC	–
✓	✓	✓	×	no FiC	–

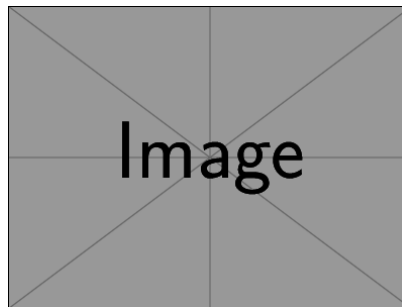
B.3 Supervised vs. Unsupervised Training We also want to examine how each loss affects the model. To this end, we see the contribution of each loss type (self/supervised contrastive loss) by allowing them to take part in model training. We run experiments showcased in table 7

Table 7: Effect of Contrastive Learning Type on Accuracy at Different Sparsity Levels.

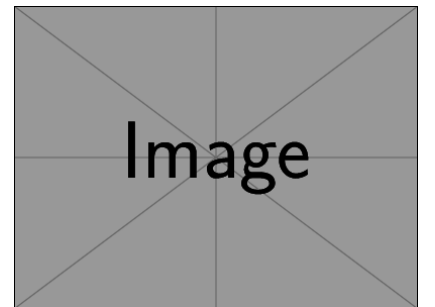
Supervised Contrastive	Self-Supervised Contrastive	Accuracy (%) at Sparsity		
		0.95	0.97	0.99
✓	✓	–	–	–
✓	×	–	–	–
×	✓	–	–	–
×	×	–	–	–



(a) Loss plot



(b) Sparsity 97%



(c) Sparsity 99%

Figure 1: Contrastive loss component plots showing accuracy vs. loss type for different sparsity levels.

B.4 Temperature Sensitivity To determine the optimal temperature τ for contrastive learning in our BaCP framework, we conducted a systematic sweep over multiple values. Table ?? reports classification accuracy on CIFAR-10 using ResNet-50 across the three target sparsity levels: 0.95, 0.97, and 0.99. All runs used the same training configuration (optimizer, learning rate, batch size, epochs) as outlined in Table 8.

Table 8: Effect of Temperature τ on Accuracy (%) across Target Sparsity Levels.

τ	Sparsity 0.95	Sparsity 0.97	Sparsity 0.99
0.03	–	–	–
0.07	–	–	–
0.15 (current)	–	–	–
0.2	–	–	–
0.3	–	–	–
0.5	–	–	–
0.7	–	–	–
1.0	–	–	–

Figure 2 below visualizes the effect of τ on accuracy. The accuracy peaks around $\tau = 0.15\text{--}0.2$, confirming the sweet spot for contrastive supervision in sparse regimes.

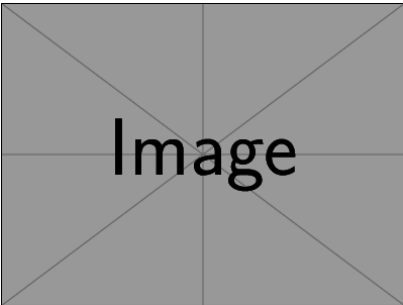


Figure 2: Temperature sensitivity plot showing accuracy vs. τ for different sparsity levels.

C. WikiText-2 Perplexity

Perplexity is a standard metric used to evaluate language models on generative tasks. It quantifies how well a model predicts the next token in a sequence, with lower perplexity values indicating better performance and higher confidence in predictions. Specifically, perplexity is computed by exponentiating the average negative log-likelihood loss over predicted tokens. For decoder-only transformer models such as RoBERTa and DistilBERT applied to masked language modeling tasks, perplexity reflects the model’s uncertainty in predicting masked tokens. Lower perplexity implies less uncertainty and thus greater confidence in the model’s predictions.

Below, we report the perplexity of RoBERTa and DistilBERT on the WikiText-2 dataset across various sparsity levels and pruning strategies:

Table 9: Perplexity Comparison at Different Sparsity Levels Across Models, Pruning Strategies, and BaCP Integration on WikiText-2 (Lower is Better)

Model	Dataset	Dense	95% Sparsity						97% Sparsity						99% Sparsity					
			Magnitude			SNIP-it			WANDA			Magnitude			SNIP-it			WANDA		
			I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP	I.P.	BaCP
DistilBERT	WikiText-2	5.743	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–
RoBERTa	WikiText-2	3.508	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–

D. Code Algorithms

D.1 Pruning Class

D.2 Iterative Pruning with WANDA

D.3 Cubic Scheduler Adapted for Epoch-Wise Pruning